Facultad de ingeniería

Docente

Leidy Mariani Dorado Flórez

Materia

Sistemas Distribuidos

Trabajo

Docker Compose Práctica

Estudiantes

Heidy Gabriela Jalvin Avirama
Jhonn Alexander Pantoja Jimenez

Popayán, Cauca 23 de marzo de 2026

TALLER PRÁCTICO – NIVEL INTERMEDIO

Sistemas Distribuidos – Diseño e Integración con Docker Compose

1. **Objetivo**

Diseñar e implementar un sistema distribuido básico utilizando Docker Compose, comprendiendo la relación entre servicios y su comunicación.

2. **Fase de diseño del sistema**

   a. Definir el tipo de sistema (ejemplo: e-commerce, app de domicilios, plataforma educativa)

   Sistema de notas

   b. Identificar mínimo tres servicios (frontend, backend, base de datos)

   i. Servicio autenticación
   ii. Servicio gestión de notas
   iii. servicio de papelera

   c. Describir cómo se comunican

   - **Servicio de Autenticación a Gestión de Notas**:
     Estos servicios tienen una comunicación síncrona ya que el servicio de notas requiere validar que el usuario es valida y tiene los permisos necesarios en tiempo real para procesar una solicitud (creación, modificación, adición o eliminación de una nota)

   - **Gestión de Notas a Papelera:**
     Cuando un usuario elimina una nota, el servicio de gestión emite un evento (Nota eliminada). El servicio de papelera escucha dicho evento y programa la eliminación definitiva sin bloquear al sistema. Por eso esta comunicación es asíncrona basada en eventos.

   d. Indicar qué ocurre si un servicio falla

   El sistema debido a que es distribuido, es tolerante a fallos, lo que indica que funcionará parcialmente sin afectar al resto de servicios.

3. **Implementación con Docker Compose**

Archivo base:

```
version: '3'
services:
web:
image: nginx
ports:
- "8080:80"
```

Completar el archivo agregando al menos un servicio adicional.

4. **Análisis**

a. ¿Cuál es el rol de cada servicio?

- **Servicio de autenticación:**
  Valida las credenciales de los usuarios en la base de datos, gestiona la sesión y permite el registro de nuevos usuarios de forma segura.

- **Servicio de Gestión de notas**:
  Permite al usuario registrado en el sistema, crear, ver, modificar y eliminar notas.

- **Servicio Papelera**:
  El rol de este servicio es tomar las notas marcadas como eliminadas y programar una eliminación definitiva de la base de datos en un determinado tiempo.

b. ¿Qué ventajas tiene dividir el sistema?

Las ventajas que tiene el dividir el sistema son las siguientes:

- **Tolerancia a fallos:**
  El fallo de un servicio cualquiera que sea, no detiene al sistema completo.
- **Escalabilidad independiente:**
  Se le pueden asignar más recursos a un servicio en específico sin tener que escalar el resto.
- **Mejora eficiencia:**
  Permite desplegar, actualizar, o reparar cada servicio por separado sin afectar al resto del sistema.

c. ¿Cómo se comunican los contenedores?

Los contenedores se comunican a través de una red interna que Docker Compose crea automáticamente. Cada servicio puede acceder a otro utilizando el nombre del servicio definido en el archivo docker-compose.yml.

5. Exploración para la siguiente clase

Agregar un servicio de base de datos (MySQL o MongoDB) en el archivo docker-compose.yml. No es necesario que funcione completamente, pero debe definirse correctamente el servicio.

1. Investigar cómo definir variables de entorno.

   Las variables de entorno en Docker Compose permiten configurar los contenedores sin modificar el código de la aplicación. Se definen en el archivo docker-compose.yml dentro de los servicios usando environment o dentro de un archivo y se envían al contenedor cuando se inicia. Se utilizan para almacenar configuraciones como usuarios, contraseñas, puertos o datos de conexión entre servicios.

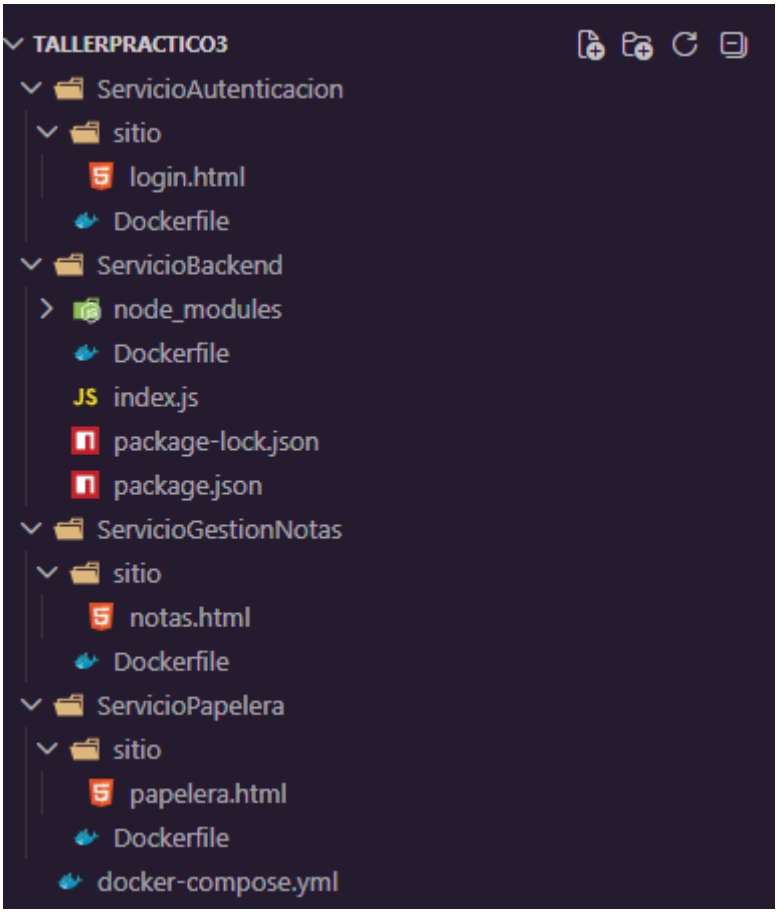
2. Buscar cómo conectar un backend con una base de datos:

   - Iniciar el proyecto y descargar dependencias (Node):
     - npm init -y
     - npm install express mysql2
     - Se debe configurar el archivo package.json con "type": "module" para usar una sintaxis más cómoda con las importaciones
     - crear el index.js
   - Vincular el backend con la base de datos previamente dockerizada:
     - En el archivo index.js , se importa createPool de mysql2. Esto permite crear la instancia de conexión trayendo los valores de las variables de entorno definidas en el Docker compose:
     -
       - host: nombre del contenedor de la base de datos
       - nombre de la base de datos
       - usuario
       - contraseña
       - puerto: El que se configuró en el docker compose para el servicio de base de datos

- Validar la conexión:
    - Se crean un endpoint específico como /pruebaconexion para ejecutar una consulta tijera. Si se resuelve con éxito devuelve un mensaje confirmando que la conexión ha sido exitosa, si falla, captura el error indicando que algo está mal configurado

3. Identificar posibles errores de conexión

- Usar localhost en el backend en lugar del nombre del servicio para conectarse a la base de datos
- El contenedor de la base de datos no está listo
- Credenciales o configuración incorrecta
- Configuración errónea de los puertos
- Variables de entorno no configuradas y usadas correctamente
- Contenedores desplegados en redes de Docker distintas

6. Entregables

a. Archivo docker-compose.yml
b. Diseño del sistema